MACHINE LEARNING ENGINEER

Remaining Useful Life (RUL) Prediction
NASA C-MAPSS Turbofan Engine Dataset

TECHNICAL WORK PACKAGE | IMPLEMENTATION BLUEPRINT | EXPERIMENTAL PROTOCOL

Member 2: classical methods, baseline, Ridge Regression, LightGBM, feature selection, tuning, comparison and error analysis.

DOCUMENT STATUS

This is a detailed, execution-ready plan and implementation guide. It does not claim models have already been trained. All performance fields must be filled using actual runs; no results are fabricated.

Deliverable	Definition of done
Runnable pipeline	Baseline, Ridge and LightGBM run using a shared data/metric protocol.
Fair comparison	Same target, split, features policy and evaluation metrics.
Reproducibility	Seeds, versions, configurations and predictions are saved.
Research-quality analysis	Metrics, diagnostic plots, feature analysis, limitations and recommendation.

1. Scope, Research Questions & Success Criteria

The work package implements classical machine-learning approaches for predicting turbofan Remaining Useful Life (RUL).

Research questions

- How much predictive value is captured by simple baselines and cycle-based features?
- Does Ridge Regression provide a stable regularized linear benchmark?
- Can LightGBM capture nonlinear effects and sensor interactions?
- Which features matter, where do errors occur, and are improvements robust across engines?

Success criteria

Criterion	Acceptance condition
Correctness	RUL labels and engine-level split verified; no test information enters training/tuning.
Fairness	All models use the same target, split boundaries and metric functions.
Reproducibility	Fixed seeds, package versions, configs and saved predictions.
Evidence	Every reported metric comes from actual predictions.
Interpretability	Comparison includes error slices and feature-importance analysis.

Role boundary

Member 2 owns baseline, Ridge, LightGBM, feature selection, tuning, model comparison and error analysis. Agree with Member 1 on data cleaning, RUL labels, split protocol and evaluation code. Member 3 owns LSTM and prediction uncertainty; use the same benchmark protocol wherever possible.

2. Dataset & Target Definition

NASA C-MAPSS contains multivariate sensor time series from simulated run-to-failure turbofan engines. The common subsets FD001–FD004 differ in operating-condition and fault-mode complexity. Confirm and document the exact subset.

Typical row structure

- Engine identifier, cycle number, operating-setting variables and sensor measurements; raw files may have no header.
- Training trajectories end at failure. Test trajectories are truncated and paired with a separate vector containing true RUL at each engine's final observed cycle.

RUL labels

For a complete training trajectory with final cycle T■, uncapped RUL at cycle t is T■ – t. A common benchmark choice caps labels at RUL_CAP: y■,t = min(T■ – t, RUL_CAP). The cap is a modeling decision; choose one value with the team and apply it consistently.

Critical test-set detail

For the official test file, the provided RUL vector describes remaining life after the final observed test cycle. Compare each engine's final-cycle prediction with its corresponding provided RUL value. Do not derive test labels from the maximum cycle within a truncated test trajectory.

Audit	Required check
Engine IDs	Count unique engines; confirm each trajectory is ordered by cycle.
Cycle sequence	Check duplicate engine-cycle rows, monotonicity and missing indices.
Missing values	Count NaN/infinite values and document treatment.
Constant sensors	Detect zero/near-zero variance using training data only.
Target integrity	For each training engine, RUL decreases by one per cycle before capping and reaches zero at its final cycle.
Test alignment	Verify supplied test RUL values map to the correct engine order.

3. Leakage-Safe Validation Protocol

Rows from the same engine are correlated. A random row-level split can put near-identical cycles in train and validation, creating optimistic estimates.

Recommended protocol

- Split by engine ID. All rows from one engine must belong to only one partition.
- Use GroupShuffleSplit or GroupKFold with engine_id as the group. For extensive tuning, use grouped CV on training engines and retain an untouched final test evaluation.
- Fit every learned transformation only on the training fold: imputation, scaling, variance filtering, feature selection and target-dependent transformations.
- Do not use official test RUL labels for hyperparameter selection. Use them once for final evaluation.
- If the team already defined a shared split protocol, follow it rather than silently changing it; record any differences.

Deployment interpretation

A held-out-engine split estimates generalization to engines unseen during training. This differs from predicting later cycles of engines already observed. State the intended scenario and make the comparison with Member 3's LSTM consistent.

Artifact	Contents
split_manifest.json	Dataset subset, seed, split method, engine IDs, target cap.
features.json	Feature list, feature-generation rules, removed sensors and scaling policy.
config.json	Parameters, search space, metric, software versions.
predictions.csv	engine_id, truth, prediction, residual, absolute error.
metrics.json	RMSE, MAE, NASA score, sample count and split identifier.

4. Feature Engineering & Selection

Start with a transparent feature set. Add complexity only when validation evidence supports it.

Feature group	Examples	Purpose / caution
Metadata	engine_id, cycle	Use engine_id for grouping; do not automatically feed identity as a predictor.
Operating settings	setting_1…setting_n	Represent operating regime and sensor variation.
Raw sensors	sensor_1…sensor_n	Core measurements; inspect constant/near-constant channels.
Cycle features	cycle, optional cycle²	Captures broad age trend; may be dataset-specific.
Rolling statistics (optional)	past-window mean/std	Must be computed within each engine using only current/past observations.
First differences (optional)	sensor(t) – sensor(t–1)	Captures local change; document first-cycle handling.
Normalized sensors (optional)	regime-adjusted sensors	Fit transformation on training data only.

Selection procedure

- 1. Remove unusable columns based on training-fold statistics only.
- 2. Keep cycle, operating settings and sensors as candidates unless a documented reason excludes them.
- 3. Inspect correlations; correlation alone is not proof a feature should be removed—Ridge tolerates correlated predictors.
- 4. Use permutation importance on held-out validation data for model-agnostic analysis. Treat LightGBM built-in importance as a secondary diagnostic.
- 5. Run feature ablations: settings + cycle, sensors only, and all approved features.

Leakage warning

Do not use the true failure cycle as a predictor. Do not compute whole-trajectory statistics that rely on future cycles when claiming online prediction. Never select features using official test labels.

5. Model A — Baseline & Model B — Ridge Regression

Baselines

Baseline	Definition	Purpose
Constant median	Predict median training RUL for every case.	Sanity-check floor robust to extreme targets.
Cycle-only model	Use cycle number (optionally a simple nonlinear transform).	Tests whether engine age alone explains much of the signal.
Minimum report	Include constant median plus main feature models.	Avoids presenting complex models without a meaningful reference.

Ridge Regression

Ridge minimizes squared error with an L2 penalty: minimize $||y - X\beta||^2 + \alpha||\beta||^2$. The penalty shrinks coefficients and stabilizes estimation when sensors are correlated. Since the penalty acts on coefficient magnitude, feature scaling is important.

- Implement StandardScaler and Ridge inside a scikit-learn Pipeline so scaling is fitted only within each training fold.
- Try log-spaced α values such as 0.01, 0.1, 1, 10, 100, 1000; expand only if the best value is at a boundary.
- Use the same split and feature policy as LightGBM where possible.
- Do not interpret standardized coefficients as causal effects; correlated sensors can share coefficient magnitude.

Illustrative implementation pattern

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import GroupKFold, GridSearchCV

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", Ridge())
])
grid = {"model__alpha": [0.01, 0.1, 1, 10, 100, 1000]}
search = GridSearchCV(
    pipe, grid, scoring="neg_root_mean_squared_error",
    cv=GroupKFold(n_splits=5), n_jobs=-1, refit=True
)
search.fit(X_train, y_train, groups=train_engine_ids)
```

The snippet assumes X_train, y_train and one group label per row are already prepared. Reduce the number of folds if there are too few engines; document the limitation.

6. Model C — LightGBM

LightGBM is a gradient-boosted decision-tree method. It can learn nonlinear relationships and interactions without feature scaling. It must still use the same leakage-safe split and metric protocol.

Parameter	Starting search	Reason
objective	regression	Continuous RUL target.
metric	rmse	Consistent primary selection metric.
n_estimators	300–1500	Use early stopping when supported.
learning_rate	0.01–0.08	Controls update size.
num_leaves	15, 31, 63	Controls tree complexity.
max_depth	-1, 5, 8, 12	Optional depth constraint.
min_child_samples	10, 20, 40, 80	Regularizes small leaves.
subsample	0.7, 0.85, 1.0	Row sampling; confirm API behavior.
colsample_bytree	0.7, 0.85, 1.0	Feature sampling.
reg_lambda	0, 1, 5, 10	L2 regularization.
random_state	fixed seed	Reproducibility.

Tuning strategy

- Begin with a small search. Tune tree complexity and regularization first; then learning rate and estimator count.
- Use grouped validation folds. If using early stopping, the validation data must be separate from the fold's training rows and never the final official test set.
- Record best parameters, fold-level scores, mean, standard deviation and fit time.
- Fix seeds and package versions; small differences can occur across versions and hardware.

Illustrative model setup

```
from lightgbm import LGBMRegressor

model = LGBMRegressor(
    objective="regression", n_estimators=700,
    learning_rate=0.03, num_leaves=31, max_depth=-1,
    min_child_samples=30, subsample=0.85,
    colsample_bytree=0.85, reg_lambda=1.0,
    random_state=42, n_jobs=-1
)
model.fit(X_train, y_train)
```

LightGBM API note: early-stopping arguments differ by version. Check the installed API and use the supported callback pattern; do not copy a version-specific snippet without testing.

7. Hyperparameter Tuning & Experiment Management

Stage	Action	Exit condition
0 — Smoke test	Fit models on a small subset; validate shapes, labels and predictions.	No pipeline errors.
1 — Baselines	Evaluate median and cycle-only models.	Reference metrics saved.
2 — Ridge	Tune α with grouped CV and a pipeline.	Best α and fold scores saved.
3 — LightGBM	Tune complexity and regularization.	Candidate range identified.
4 — Refine	Tune learning rate / estimator count; early stop if supported.	Final validation configuration.
5 — Robustness	Inspect fold variability and run feature ablations.	Stability evidence.
6 — Lock	Freeze features, cap, split, parameters and metrics.	Versioned config saved.
7 — Final test	Evaluate once on official test set.	Final predictions and metrics saved.

Every experiment should log

run_id, timestamp, subset, target cap, split seed/method, feature-set version, model, parameter dictionary, package versions, train/validation counts, RMSE, MAE, NASA score, fit time, prediction path and notes.

Selection metric

Use validation RMSE as the primary tuning metric unless the team agrees otherwise before examining results. Also report MAE and the NASA asymmetric score if required. The model with best RMSE may not have the best NASA score because late predictions can receive a larger penalty.

8. Metrics & Correct Implementation

All metrics must use the same evaluation rows and RUL units (cycles).

Metric	Definition	Interpretation
MAE	mean(|y – ■|)	Average absolute error in cycles.
RMSE	sqrt(mean((y – ■)²))	Penalizes large errors more strongly.
NASA score	Asymmetric exponential penalty sum.	Late predictions are typically penalized more; document exact formula.
Bias	mean(■ – y)	Positive = overprediction; negative = underprediction.
P90 absolute error	90th percentile of |y – ■|	Tail-risk indicator.

Residual convention

Define residual = prediction – truth. A positive residual means the model predicts more remaining life than actually remains—a late/optimistic prediction that can be operationally risky.

Illustrative metric functions

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error

def regression_metrics(y_true, y_pred):
    err = np.asarray(y_pred) - np.asarray(y_true)
    return {
        "MAE": float(mean_absolute_error(y_true, y_pred)),
        "RMSE": float(np.sqrt(mean_squared_error(y_true, y_pred))),
        "Bias": float(np.mean(err)),
        "P90_AE": float(np.percentile(np.abs(err), 90)),
    }

def nasa_score(y_true, y_pred):
    d = np.asarray(y_pred) - np.asarray(y_true)
    penalty = np.where(d < 0, np.exp(-d / 13.0) - 1,
                       np.exp(d / 10.0) - 1)
    return float(np.sum(penalty))
```

NASA-score caution

The shown formula is a common implementation pattern, not a substitute for the course's agreed benchmark definition. Verify sign conventions, constants, and whether scores are summed or averaged. Exponential penalties can become very large; check overflow and units.

9. Model Comparison & Error Analysis

Results table — fill only after real runs

Model	Feature set	Val RMSE	Val MAE	NASA score	Fit time	Notes
Median baseline	None	—	—	—	—	Measured values
Cycle-only	Cycle	—	—	—	—	Measured values
Ridge	Approved set	—	—	—	—	Best α
LightGBM	Approved set	—	—	—	—	Best parameters

Required diagnostic plots

- Predicted vs. true RUL scatter plot with y = x reference line.
- Residual distribution histogram; inspect skew and long tails.
- Absolute error vs. true RUL; check whether error changes near failure or at high RUL.
- Per-engine ranked error plot/table; identify engines dominating total error.
- Validation permutation importance; LightGBM built-in importance as secondary evidence.
- Ablation comparison: settings + cycle, sensors only, all approved features.

Analysis questions

- Does LightGBM improve both RMSE and MAE, or only reduce a few large errors?
- Does the model systematically overpredict RUL near failure?
- Are errors linked to operating regimes or particular sensor patterns?
- Does target capping affect errors at high RUL?
- Are improvements consistent across grouped folds, or smaller than fold-to-fold variation?
- Are important features stable when correlated features are considered?

No fabricated conclusions

Do not claim LightGBM is superior, a specific sensor is most important, or a cap is optimal until measured results support the statement.

10. Repository Structure & Engineering Standards

Suggested project layout

```
project/
  data/                 # raw data; keep immutable
  notebooks/
    01_data_audit.ipynb
    02_model_comparison.ipynb
  src/
    data_loader.py      # parsing and column names
    labels.py           # RUL target construction
    features.py         # feature selection/engineering
    split.py             # engine-level split manifest
    models.py            # baseline, Ridge, LightGBM
    metrics.py           # RMSE, MAE, NASA score
    train_eval.py        # shared evaluation pipeline
    plots.py             # diagnostic figures
  configs/experiment.json
  outputs/metrics.csv
  outputs/predictions/
  outputs/figures/
  README.md
```

Engineering rules

- Keep raw data immutable; generate processed data deterministically.
- Use explicit feature lists rather than positional column selection after transformations.
- Assert train and validation engine IDs do not overlap.
- Assert truth and prediction lengths match and predictions contain no NaN/infinity.
- Save fitted model, feature manifest, configuration and row-level predictions.
- Record tested package versions in requirements.txt or an environment file.
- Generate tables and figures from saved prediction files, not manually copied numbers.

11. Failure Modes & Quality Controls

Failure mode	Why it matters	Control
Random row split	Leakage between related engine cycles.	Group by engine ID.
Scaling before split	Validation distribution leaks into preprocessing.	Use pipeline/fold-local fitting.
Tuning on test labels	Biased final estimate.	Lock test set until final evaluation.
Wrong test RUL mapping	Truths paired with wrong predictions.	Preserve engine mapping and verify order.
Different target caps	Models solve different tasks.	One shared label function and cap.
Different metric code	Scores are not comparable.	One shared evaluation module.
Future-derived features	Overstates online prediction ability.	Use current/past observations only.
Selective reporting	Can bias conclusions.	Keep a run log and explain exclusions.
Only one metric	Can hide bias or tail failures.	Report RMSE, MAE, NASA score, bias and plots.

Pre-submission checklist

- Dataset subset and RUL cap are documented.
- Label function has been sanity-checked on a complete training trajectory.
- Train/validation engine overlap is zero.
- Baseline, Ridge and LightGBM use shared metrics.
- Official test labels were not used for tuning.
- Best parameters and software versions are recorded.
- Figures and tables are generated from saved predictions.
- Limitations and future work are explicit.

12. Deliverables, Timeline & Defense Preparation

Deliverable	Contents	Acceptance check
01 — Model code	Baseline, Ridge pipeline, LightGBM training/evaluation.	Runs in documented environment.
02 — Feature spec	Feature columns, engineering, filtering and leakage notes.	Recreates exact feature matrix.
03 — Tuning log	Search space, folds, best params and fold scores.	No test-based selection.
04 — Results table	Validation and final-test metrics.	Shared metric definitions.
05 — Prediction files	Engine ID, truth, prediction and residual.	Rows align to the protocol.
06 — Diagnostic figures	Prediction, residual, error-vs-RUL, feature importance.	Readable labels and captions.
07 — Report section	Methods, formulas, configuration, results and limitations.	Claims supported by outputs.
08 — Defense notes	Rationale, metrics, leakage controls and failure cases.	Member can explain trade-offs.

Suggested milestones

Milestone	Work	Exit condition
M1	Confirm subset, target cap, split and metrics with team.	Written protocol.
M2	Audit data, labels, metrics and baseline.	Sanity checks pass.
M3	Build Ridge and grouped tuning.	Ridge results saved.
M4	Train/tune LightGBM and ablations.	Validation selection complete.
M5	Complete diagnostics and error analysis.	Evidence-backed interpretation.
M6	Final evaluation and defense prep.	Frozen code/config/report.

Defense questions

- Why use Ridge if LightGBM is more flexible? It is a regularized, interpretable benchmark and tests whether complexity is justified.
- Why not randomly split rows? Engine cycles are correlated; row splitting leaks trajectory-specific information.
- Why RMSE and MAE? RMSE emphasizes large errors; MAE communicates typical absolute error.
- Why can NASA score rank models differently? It uses asymmetric penalties and can penalize late RUL predictions more.
- Why not use engine ID as a predictor? It can encode arbitrary identity instead of transferable degradation.
- What is the main limitation? C-MAPSS is simulated and benchmark-specific; results may not transfer directly to real fleet operations.

Final handoff

The package is complete when another teammate can reproduce the baseline, Ridge and LightGBM results from the documented data/configuration, verify that no engine leakage occurred, inspect saved predictions and plots, and explain the final model choice without unsupported claims.